Problem:

For this project, we will be exploring publicly available data from LendingClub.com. Lending Club is a platform that connects people who need money (borrowers) with people who have money (investors). As an investor, your primary goal is to minimize risk—you want to invest exclusively in individuals who show a high probability of paying you back!

Understanding Decision Trees 🌳

To Understand the Decision Trees, I suggest you watch this youtube video. That's How I learned it😁😁

Decision Trees - VisuallyExplained

Answer Code:

loan_data.csv | Decision Trees & Random Forest.py

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

#Read the CSV file
loans = pd.read_csv('loan_data.csv')

#Check the data set
print('___'*500)
print(loans.head())
print('___'*500)
print(loans.info())
print('___'*500)
print(loans.describe())
print('___'*500)
print()
```

Initial Data Exploration: Getting to Know Our Dataset 🔍

Since you guys have been following these notes carefully, you probably already know exactly what I am doing in this first block of code! Before we even think about training a model, I am basically just playing around with the data to see what it looks like.

We need to answer a few basic questions: What kind of values do we have? Are they text (strings), whole numbers (integers), or decimals (floats)? Having a crystal clear idea of our data types is a mandatory first step.

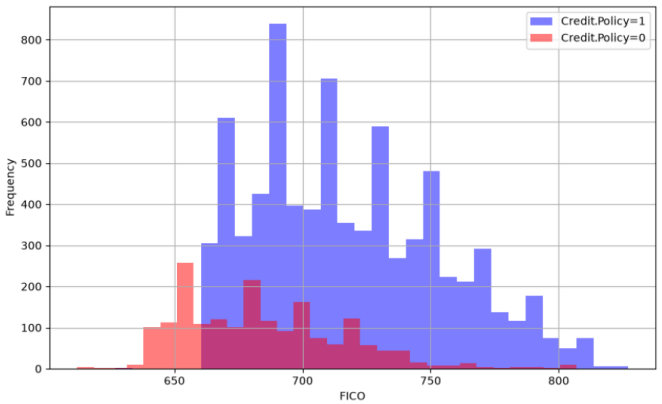
Here is a quick breakdown of the code from `image_f9b046.png`:

- Line 10 (`pd.read_csv`): We load the Lending Club data from our CSV file into a pandas DataFrame named `loans`.
- Line 14 (`loans.head()`): We print the first 5 rows to get a visual feel for the columns.
- Line 16 (`loans.info()`): This is the most crucial step right now. It tells us the data type of every single column.
- Line 18 (`loans.describe()`): This runs quick math (like averages and maximums) on all our numerical columns to help us spot any weird outliers.

Exploratory Data Analysis

Let's do some data visualization! We'll use seaborn and pandas built-in plotting capabilities, but feel free to use whatever library you want. Don't worry about the colors matching, just worry about getting the main idea of the plot.

```
22
23    plt.figure(figsize=(10,6))
24    loans[loans['credit.policy']==1]['fico'].hist(alpha=0.5,color='blue',bins=30,label='Credit.Policy=1')
25    loans[loans['credit.policy']==0]['fico'].hist(alpha=0.5,color='red', bins=30,label='Credit.Policy=0')
26    plt.legend()
27    plt.xlabel('FICO')
28    plt.show()
```



Analyzing the FICO Score Graph 📊

We plotted this histogram to figure out exactly how a borrower's FICO credit score affects their chances of getting a loan approved.

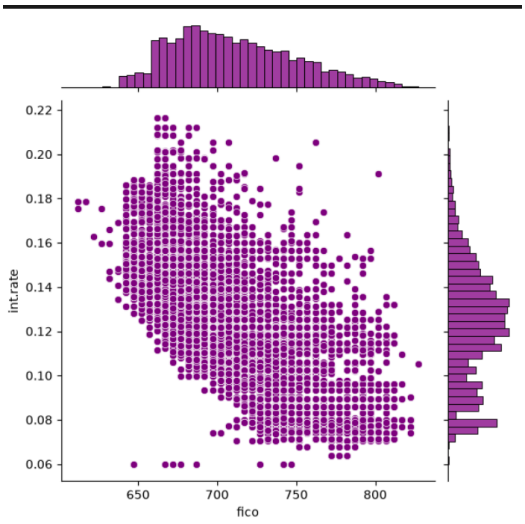
Reading the Graph

- **X-Axis:** Represents the actual FICO score.
- **Y-Axis:** Represents the total number of people (the headcount) who have that specific score.
- **Blue Bars:** Represent the people who *were approved* for the loan (Credit Policy = 1).
- **Red Bars:** Represent the people who *were rejected* for the loan (Credit Policy = 0).

Conclusion & Key Takeaways 🎯 After reviewing this graph, we can uncover a few hidden rules about how LendingClub operates:

- **The 660 Cutoff:** You can clearly see that people with a FICO score below roughly 660 face an automatic rejection. In that lower section of the graph, there are absolutely no blue bars—only the red bars of rejected applicants!
- **The Approval Zone:** The vast majority of the accepted borrowers belong to the massive blue mountains on the right side. Every single person who was accepted for a loan sits strictly above that 660 mark.
- **The Overlap Mystery:** If you look very carefully, you will notice some red bars mixed in with the blue bars in the higher score ranges (above 660). This proves that having a good FICO score does *not* guarantee an approval. Why would someone with a high score get rejected? This happens because a loan decision isn't based on a credit score alone. Even if your FICO score is excellent, you can still face rejection due to other negative financial factors in the dataset, such as a low annual income, a dangerous debt-to-income ratio, or a history of public bankruptcy.

```
30
31    sns.jointplot(x='fico',y='int.rate',data=loans,color='purple')
32    plt.show()
```



Understanding the Code

- `sns.jointplot(...)`: We are using the Seaborn library (`sns`) to create a "joint plot." This is an incredibly useful visualization because it combines three completely different graphs into one single image.
- `x='fico', y='int.rate'`: We are explicitly telling the graph to place the FICO score on the horizontal x-axis and the Interest Rate on the vertical y-axis.
- `data=loans, color='purple'`: We point the function to our `loans` dataset and set the color to purple to keep it visually distinct from our previous blue and red histogram.

Reading the Graph

When you look at "image_fb0ced.png", you are actually looking at three distinct pieces of data:

- The Center (The Scatter Plot): The massive cluster of purple dots shows the direct relationship between a person's FICO score and their assigned interest rate. Every single dot represents an individual borrower.
- The Top (FICO Histogram): The bar chart running along the top roof of the graph shows the headcount distribution of FICO scores.
- The Right Side (Interest Rate Histogram): The bar chart running down the right wall shows the frequency of different interest rates. You can see the largest spikes happen around the 0.11 to 0.14 (11% to 14%) range.

Conclusion & Key Takeaways

After reviewing the central scatter plot, the core mathematical rule of money lending becomes instantly visible: As your FICO score increases, your interest rate decreases.

- The High-Risk Zone: Look at the left side of the scatter plot (lower FICO scores around 650-670). The dots are pushed very high up on the y-axis. This means borrowers with lower credit scores are being hit with massive interest rates, sometimes reaching up to 0.22 (22%).
- The Safe Zone: Now follow the cluster to the right side (high FICO scores around 750-800). The dots drop completely to the floor of the graph, settling down near 0.06 to 0.08 (6% to 8%).

```
38    cat_feats = ['purpose']
39
```

Remember!! We did the same thing in our Linear Regression part, Scikit-Learn models only understand numbers? We discovered that the `purpose` column in our dataset is completely filled with raw text categories (words like "credit_card", "debt_consolidation", and "educational"). If we feed this text directly into our Decision Tree, the entire algorithm will crash.

To fix this, we need to perform **Feature Engineering**—specifically, we need to convert these text categories into numerical "dummy variables" (a series of 1s and 0s). The line of code in `image_fb1830.png` is the very first step in that transformation process.

- **The Variable:** We create a simple Python list and name it `cat_feats` (an abbreviation for "categorical features").
- **The Content:** Inside this list, we store the exact string name of the column that contains the text we need to fix: `['purpose']`.
- **The Logic:** In the next step of our code, we are going to use a special pandas command to mathematically transform our data. Instead of hard-coding the column name directly into that complex transformation function, we store it in this list first. We can then hand this list to pandas and say, *"Hey, look inside this list and transform any column you find in there into numbers!"*

This approach keeps our code incredibly clean and flexible. If we ever expand our dataset in the future and end up with five different text columns that all need fixing, we won't have to rewrite our complex transformation logic. We just add those new column names directly into this single `cat_feats` list!

```
39
40    final_data = pd.get_dummies(loans, columns=cat_feats, drop_first=True, dtype=int)
41
```

Transforming Text into Numbers 🪄

This line of code is where the actual Feature Engineering magic happens. We are officially using pandas to translate our raw text categories into strict mathematical numbers.

Here is the step-by-step breakdown of the code in "image_fb1fd1.png":

- `final_data =`: We are creating a brand new dataset (DataFrame) to hold our updated information. This leaves our original `loans` dataset completely untouched just in case we make a mistake.
- `pd.get_dummies()`: This is the specific pandas tool that converts categorical text data into numerical "dummy variables." It takes a column filled with words and splits it into several new columns filled with 1s and 0s (True or False).
- `loans`: We are telling the function to apply its changes to our main dataset.

- `columns=cat_feats`: Remember the list we just created in the previous step? We pass it in right here. This tells pandas to only apply the dummy transformation to the specific columns listed inside `cat_feats` (which, right now, is just our `purpose` column).
- `drop_first=True`: This is a critical statistical safety measure used to avoid something called the "Dummy Variable Trap."

Understanding the Dummy Variable Trap (`drop_first=True`) 🪤

To understand why we must drop a column, imagine our `purpose` column only had three options: *Credit Card*, *Education*, and *Small Business*.

When pandas creates the dummy variables, it naturally wants to create three separate columns:

1. Is_Credit_Card (1 or 0)
2. Is_Education (1 or 0)
3. Is_Small_Business (1 or 0)

However, machine learning algorithms are incredibly smart. If the algorithm looks at a row of data and sees that `Is_Education` is 0 and `Is_Small_Business` is 0, it automatically deduces that the purpose *must* be a Credit Card.

Because that first column (`Is_Credit_Card`) doesn't provide any new mathematical information, keeping it actually confuses the algorithm and messes up the model's math (an issue known as multi-collinearity). By setting `drop_first=True`, we instantly delete that redundant first column, keeping our data perfectly optimized for our Decision Tree!

```
42    print(final_data.info())
```

Use this and see what happens to our dataset now.

Predictions and Evaluation of Decision Tree

```
47
48  from sklearn.model_selection import train_test_split
49
50  X = final_data.drop('not.fully.paid',axis=1)
51  y = final_data['not.fully.paid']
52  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=101)
53
54  from sklearn.tree import DecisionTreeClassifier
55  dtree = DecisionTreeClassifier()
56  dtree.fit(X_train, y_train)
57  predictions = dtree.predict(X_test)
58
59  from sklearn.metrics import classification_report,confusion_matrix
60  print('___'*500)
61  print(classification_report(y_test,predictions))
62  print('___'*500)
63  print(confusion_matrix(y_test,predictions))
64  print('___'*500)
```

As we have done in our previous projects, we trained the model and are now evaluating its predictive power using our standard machine learning workflow. The only major difference this time is that we have swapped out our previous equations for a flowchart-based algorithm.

- Defining the Input and Output (Lines 50-51): We define our target variable `y` by isolating the `not.fully.paid` column (the answer key indicating if the loan defaulted). We define our features `X` by dropping the `not.fully.paid` column from `final_data`, leaving only the predictive clues behind.
- The 70/30 Split (Line 52): Using `train_test_split`, we divide our dataset. We hand 70% of the data to the model for training and hide the remaining 30% (`test_size=0.30`) to use as the final exam. Setting `random_state=101` ensures the data is shuffled the exact same way every time we run the script.
- Building the Brain (Lines 57-58): We import the `DecisionTreeClassifier` from Scikit-Learn and create a blank model named `dtree`.
- Studying and Testing (Lines 59-60): We use `dtree.fit(X_train, y_train)` to force the model to study the historical loans and build its internal "20 Questions" flowchart. Immediately after, we use `dtree.predict(X_test)` to administer the final exam, forcing it to guess the outcomes of the 30% hidden data.
- The Report Card (Lines 63-68): We import our grading tools. The `classification_report` will print our standard metrics (Precision, Recall, F1-Score, and Accuracy) just like it did in our Logistic Regression project. We also added a `confusion_matrix`, which prints a simple 2x2 grid showing the exact number of True Positives, True Negatives, False Positives, and False Negatives the model guessed.

Now lets evaluate these outputs

```
--------------------------------------------------------------
              precision    recall  f1-score   support

           0       0.86      0.82      0.84      2431
           1       0.20      0.24      0.22       443

    accuracy                           0.73      2874
   macro avg       0.53      0.53      0.53      2874
weighted avg       0.75      0.73      0.74      2874


--------------------------------------------------------------
[[1993  438]
 [ 335  108]]
--------------------------------------------------------------
```

Our Decision Tree has taken its final exam. The output reveals a critical lesson about machine learning: a decent overall accuracy score can sometimes hide a model's true weaknesses.

The Confusion Matrix (The 2x2 Grid)

The matrix at the bottom breaks down exactly how the model guessed on the 2,874 test loans.

- **1993 (True Negatives):** The model correctly predicted these borrowers would pay back their loan in full (Class 0).
- **108 (True Positives):** The model correctly predicted these borrowers would default on their loan (Class 1).
- **438 (False Positives):** The model incorrectly flagged good borrowers as defaulters.
- **335 (False Negatives):** The most dangerous mistake for an investor! The model thought these people were safe to lend to, but they actually defaulted and lost the investor's money.

The Classification Report: Safe Borrowers (Class 0)

- **Support:** Out of the hidden test group, 2,431 people actually paid back their loans.
- **Precision (86%) & Recall (82%):** When predicting someone is a safe bet, the tree is highly reliable. It successfully identified 82% of all the actual good borrowers in the dataset.

The Classification Report: Defaulters (Class 1)

- **Support:** Only 443 people in our test group actually defaulted.
- **Precision (20%):** When the model flags a borrower as a defaulter, it is only correct 20% of the time. The other 80% of the time, it is falsely accusing a perfectly safe borrower.
- **Recall (24%):** Out of the 443 actual defaulters, the model only successfully caught 108 of them. It completely missed the remaining 76%, labeling them as safe.

The Hidden Trap: Data Imbalance

The overall Accuracy sits at 73%, which sounds acceptable on a surface level. However, this model is performing terribly at its primary objective: protecting investors from bad loans.

This failure is directly caused by an imbalanced dataset. Because there are vastly more people who pay back their loans (2,431) than people who default (443) in this specific CSV, the Decision Tree simply did not have enough historical examples of "bad borrowers" to learn what a true default profile looks like. It over-optimized for the majority class and essentially failed to understand the minority class.

Now let’s go to the Solution for this low accuracy. It’s Forest Model

```python
# Training the Random Forest model
#==============================================================================

from sklearn.ensemble import RandomForestClassifier

rfc = RandomForestClassifier(n_estimators=600, random_state=101)
rfc.fit(X_train, y_train)

rfc_predictions = rfc.predict(X_test)

from sklearn.metrics import classification_report, confusion_matrix

print('___'*100)
print("RANDOM FOREST CLASSIFICATION REPORT")
print('___'*100)
print(classification_report(y_test, rfc_predictions))

print('___'*100)
print("RANDOM FOREST CONFUSION MATRIX")
print(confusion_matrix(y_test, rfc_predictions))
print('___'*100)
```

```
------------------------------------------------------------------------------------------------------
              precision    recall  f1-score   support

           0       0.85      1.00      0.92      2431
           1       0.52      0.03      0.05       443

    accuracy                           0.85      2874
   macro avg       0.69      0.51      0.48      2874
weighted avg       0.80      0.85      0.78      2874


------------------------------------------------------------------------------------------------------
RANDOM FOREST CONFUSION MATRIX
[[2420   11]
 [ 431   12]]
```

Analyzing the Random Forest Results

- **The Accuracy Trap in Action:** At first glance, the overall Accuracy jumped to 85% (up from the single Decision Tree's 73%). If you only looked at that one number, you might think the Random Forest is a massive upgrade. But the breakdown tells a terrifying story for our investor.

- **The Confusion Matrix Breakdown:**
    - **2420 (True Negatives):** The model successfully identified almost all the good borrowers.
    - **12 (True Positives):** Out of 443 total bad borrowers, the forest only successfully caught 12!
    - **431 (False Negatives):** The model completely missed 431 bad loans, confidently telling the investor they were perfectly safe to fund.
- **The Recall Collapse:** Look at the Recall column for Class 1 (Defaulters). It plummeted from 24% in our single tree down to a dismal 3% (0.03) here.
- **Why did 600 trees perform worse at finding bad loans?** This perfectly illustrates the danger of imbalanced data. Because roughly 85% of the historical dataset represents safe borrowers, the Random Forest mathematically optimized for the majority. When you have 600 trees voting on highly skewed data, the sheer volume of "Safe" examples drowns out the subtle clues of a "Defaulter." The forest basically learned that if it just guesses "Safe" almost every single time, it will automatically score an 85% on the test!

While the Random Forest looks mathematically superior on paper, the single Decision Tree was actually much more valuable for our specific scenario. The single tree caught 108 bad loans, whereas the mighty forest only caught 12. In the real world of engineering physics, finance, and machine learning, you must always align your evaluation metrics with your final objective—sometimes a lower overall accuracy produces a far safer real-world tool.